Task 1: Create a Docker Image from a Running Container

1. Objective
To create a custom Docker image from a running container using the docker commit command.

2. Purpose
To understand how changes made inside a running Docker container can be saved as a new image and reused to create additional containers.

3. Environment
Cloud Platform: AWS
Compute Service: Amazon EC2
Operating System: Amazon Linux
Container Platform: Docker
Base Image: nginx

4. Implementation Steps

Step 1: Verify Docker

```
docker –version
docker info
```

Step 2: Pull the Base Image

```
docker pull nginx:alpine
```

Step 3: Create and Run the Container

```
docker run -d \
  --name task1-container \
  -p 8080:80 \
  nginx:alpine
```

Step 4: Customize the Container

```
docker exec task1-container sh -c \

'echo "<h1>Docker Image Created from Running Container</h1><p>Task 1 completed on Amazon Linux EC2</p>" > /usr/share/nginx/html/index.html'
```

Step 5: Validate the Changes

```
curl http://localhost:8080
```

Step 6: Create a Custom Image

```
docker commit task1-container docker-lab/task1-web:v1
```

Step 7: Verify the Image

```
docker images
```

Step 8: Run a New Container from the Custom Image

```
docker run -d \
  --name task1-test \
  -p 8081:80 \
  docker-lab/task1-web:v1
```

Step 9: Validate the New Container

curl http://localhost:8081

5. Key Commands Explained

docker pull: Downloads an image from a registry.
docker run: Creates and starts a container.
docker exec: Executes a command inside a running container.
docker commit: Creates an image from a container's changes.
docker images: Lists locally available images.
curl: Tests the HTTP response from the application.
docker ps: Lists running containers.

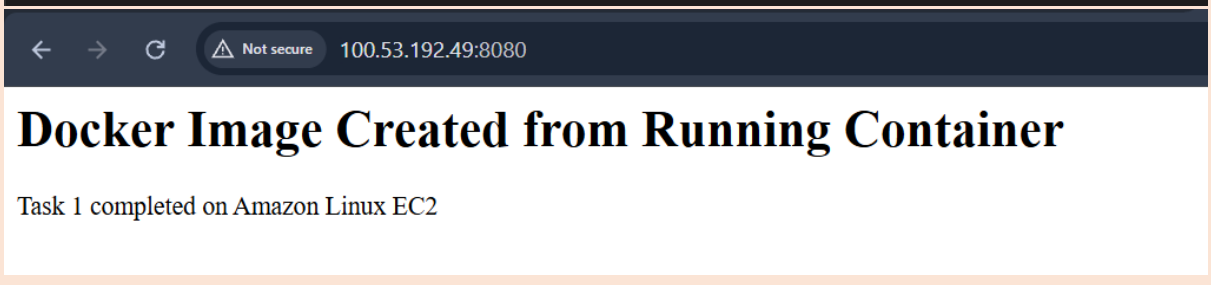
5.Implementation

```
[root@ip-172-31-30-255 ec2-user]# docker images
REPOSITORY   TAG       IMAGE ID       CREATED       SIZE
nginx        alpine    3dd08163706a   3 weeks ago   62.9MB
[root@ip-172-31-30-255 ec2-user]# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS          PORTS                                     NAMES
93d485252c5a   nginx:alpine   "/docker-entrypoint.…"   15 seconds ago   Up 14 seconds   0.0.0.0:8080->80/tcp, :::8080->80/tcp   task1-container
[root@ip-172-31-30-255 ec2-user]# curl http://localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

```
[root@ip-172-31-30-255 ec2-user]# docker exec task1-container sh -c \
'echo "<h1>Docker Image Created from Running Container</h1><p>Task 1 completed on Amazon Linux EC2</p>" > /usr/share/nginx/html/index.html'
[root@ip-172-31-30-255 ec2-user]# curl http://localhost:8080
<h1>Docker Image Created from Running Container</h1><p>Task 1 completed on Amazon Linux EC2</p>
[root@ip-172-31-30-255 ec2-user]# docker commit task1-container docker-lab/task1-web:v1
sha256:411adc30e1313ddf3be82432e722db27c833d01c7e13677862ab0e3c90f16195
[root@ip-172-31-30-255 ec2-user]# docker images
REPOSITORY            TAG       IMAGE ID       CREATED          SIZE
docker-lab/task1-web  v1        411adc30e131   10 seconds ago   62.9MB
nginx                 alpine    3dd08163706a   3 weeks ago      62.9MB
[root@ip-172-31-30-255 ec2-user]# docker run -d \
  --name task1-test \
  -p 8081:80 \
  docker-lab/task1-web:v1
6989170d6832f48e650b600e663292790db4ffc7d3f4b0a7250ac4147a855faa
[root@ip-172-31-30-255 ec2-user]# docker ps
CONTAINER ID   IMAGE                    COMMAND                  CREATED              STATUS              PORTS                                     NAMES
6989170d6832   docker-lab/task1-web:v1  "/docker-entrypoint.…"   6 seconds ago        Up 6 seconds        0.0.0.0:8081->80/tcp, :::8081->80/tcp   task1-test
93d485252c5a   nginx:alpine             "/docker-entrypoint.…"   About a minute ago   Up About a minute   0.0.0.0:8080->80/tcp, :::8080->80/tcp   task1-container
[root@ip-172-31-30-255 ec2-user]# curl http://localhost:8081
<h1>Docker Image Created from Running Container</h1><p>Task 1 completed on Amazon Linux EC2</p>
[root@ip-172-31-30-255 ec2-user]#
```



6. Conclusion

Successfully created a custom Docker image from a running NGINX container on Amazon Linux EC2 using docker commit. Verified that the image could be reused to launch another container with the saved customization.

Cleanup is deferred until all eight hands-on tasks are completed.

Task 2: Copy a Docker Image from EC2 to a Local Machine and Load It Back

1. Objective
To transfer a Docker image from an Amazon Linux EC2 instance to a Windows laptop and load the image back into Docker.

2. Purpose
To understand how Docker images can be saved as portable TAR archives, transferred between systems using SCP, and imported into Docker without rebuilding the image.

3. Environment

Cloud Platform: AWS
Source/Docker Server: Amazon Linux EC2
Server IP: 100.53.192.49
Local Machine: Windows
Terminal: Git Bash
SSH Private Key: Devops.pem
Docker Image: docker-lab/task1-web

4. Implementation Steps

Step 1: Verify the Docker Image

```
docker images
```

The image docker-lab/task1-web:v1 was available on EC2.

Step 2: Save the Docker Image as a TAR Archive

```
docker save -o /home/ec2-user/task1-web.tar docker-lab/task1-web:v1
```

The archive was created with a size of approximately 62 MB.

Step 3: Correct the Archive Permissions
The archive was initially owned by root with permissions that prevented the SSH user from reading it.
The following commands were executed on EC2:

```
sudo cp /home/ec2-user/task1-web.tar /tmp/task1-web.tar
sudo chown ec2-user:ec2-user /tmp/task1-web.tar
chmod 644 /tmp/task1-web.tar
```

The archive was then readable by the SSH user.

Step 4: Copy the Archive to Windows Using Git Bash

```
scp -i "/c/Users/Samala Sneha/Downloads/Devops.pem" \
ec2-user@100.53.192.49:/tmp/task1-web.tar \
"/c/Users/Samala Sneha/Downloads/task1-web.tar"
```

The transfer completed successfully. The TAR file was present in the Windows Downloads directory.

Step 5: Verify the Local Archive

```
ls -lh "/c/Users/Samala Sneha/Downloads/task1-web.tar"
```

The archive was approximately 62 MB.

Step 6: Load the Image into Docker

On the EC2 server: docker load -i /tmp/task1-web.tar
Output: Loaded image: docker-lab/task1-web:v1

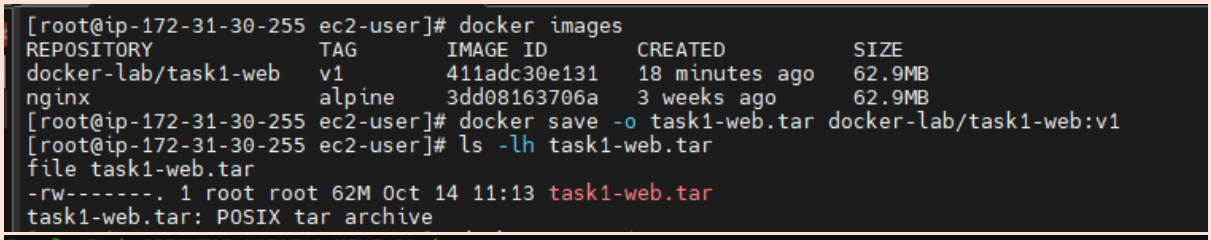
Step 7: Verify the Loaded Image

docker images
The image docker-lab/task1-web:v1 was present after loading.

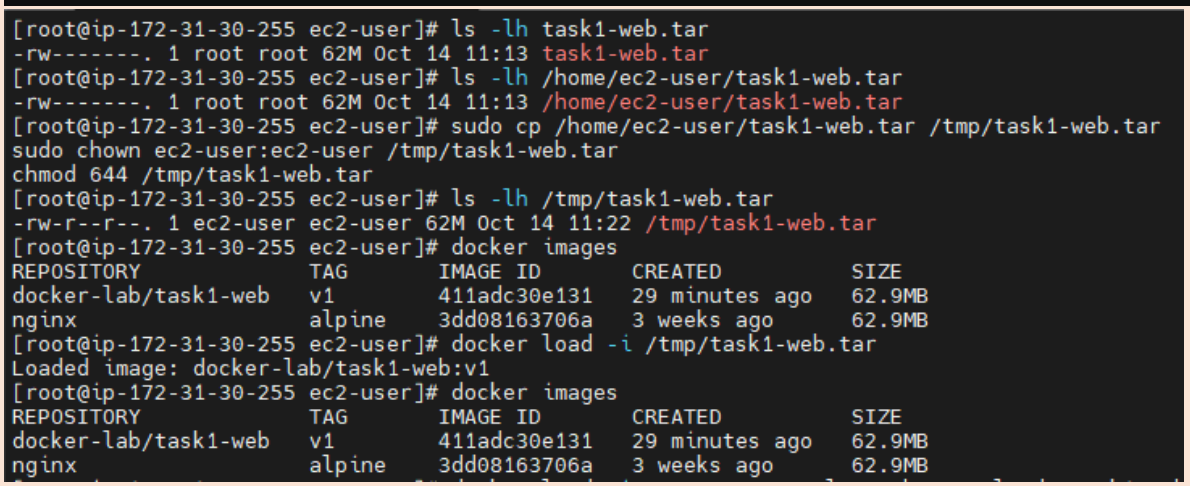
5. Key Commands Explained

docker save: Packages a Docker image into a TAR archive.
scp: Securely copies files between systems over SSH.
chown: Changes file ownership.
chmod: Changes file permissions.
docker load: Imports an image archive into Docker.
docker images: Lists local Docker images.

6. Implementtaion







7. Conclusion

Successfully transferred a Docker image archive from Amazon Linux EC2 to a Windows laptop using Git Bash and SCP, then loaded the image back into Docker using docker load.

Task 3: Create a Docker Image Using Alpine Linux and Customize It with Tomcat

1. Objective
To build a custom Docker image using Alpine Linux as the base operating system, install Java and Apache Tomcat, and deploy a sample web application.

2. Purpose
This task demonstrates how to build custom container images using a Dockerfile, install application dependencies, configure a Java application server, and run a web application inside a Docker container.

3. Environment

Cloud Platform: AWS
Compute Service: Amazon EC2
Operating System: Amazon Linux
Container Platform: Docker
Base Image: Alpine Linux 3.22
Java Runtime: OpenJDK 17 JRE
Application Server: Apache Tomcat 10.1.61
Custom Image: alpine-tomcat:1.0
Container Name: task3-tomcat
Port Mapping: EC2 port 8082 to container port 8080

4. Implementation Steps

Step 1: Create the Project Directory

```
mkdir -p ~/task3-alpine-tomcat
cd ~/task3-alpine-tomcat
```

Step 2: Create the Sample Web Page

```
cat > index.html <<'EOF'
<!DOCTYPE html>
<html>
<head>
  <title>Alpine Tomcat Docker Lab</title>
</head>
<body>
  <h1>Task 3 Completed Successfully!</h1>
  <p>Tomcat is running inside a custom Alpine Linux Docker image.</p>
  <p>Platform: AWS EC2 - Amazon Linux</p>
</body>
</html>
EOF
```

Step 3: Create the Dockerfile

```
FROM alpine:3.22

LABEL maintainer="DevOps Lab"
LABEL purpose="Custom Alpine image with Apache Tomcat"

ARG TOMCAT_VERSION=10.1.61

RUN apk add --no-cache openjdk17-jre curl tar \
   && mkdir -p /opt/tomcat \
   && curl -fSL \
     "https://downloads.apache.org/tomcat/tomcat-10/v${TOMCAT_VERSION}/bin/apache-tomcat-
${TOMCAT_VERSION}.tar.gz" \
     -o /tmp/tomcat.tar.gz \
   && tar -xzf /tmp/tomcat.tar.gz \
     -C /opt/tomcat --strip-components=1 \
   && rm /tmp/tomcat.tar.gz \
   && apk del curl tar \
   && addgroup -S tomcat \
   && adduser -S -D -H -G tomcat tomcat \
   && chown -R tomcat:tomcat /opt/tomcat

COPY index.html /opt/tomcat/webapps/ROOT/index.html

EXPOSE 8080

USER tomcat

CMD ["/opt/tomcat/bin/catalina.sh", "run"]
```

Step 4: Build the Custom Image

```
docker build -t alpine-tomcat:1.0 .
```

Step 5: Verify the Image

```
docker images
```

Step 6: Run the Tomcat Container

```
docker run -d \
  --name task3-tomcat \
  -p 8082:8080 \
  alpine-tomcat:1.0
```

Step 7: Validate the Container

```
docker ps
docker logs task3-tomcat
```

Step 8: Test the Application

curl http://localhost:8082

The application should return the custom HTML page.

Step 9: Optional Browser Access

http://YOUR_EC2_PUBLIC_IP:8082

5. Key Commands Explained

FROM: Selects the base image.
RUN: Executes commands during image construction.
COPY: Copies files into the image.
EXPOSE: Documents the container's application port.
USER: Sets the user used to run the application.
CMD: Defines the default command when the container starts.
docker build: Builds a Docker image from a Dockerfile.
docker run: Creates and starts a container.
docker logs: Displays container logs.
curl: Tests the HTTP response.

6. Implementation

```
nginx                 alpine    3dd08163706a   3 weeks ago      62.9MB
[ec2-user@ip-172-31-30-255 task3-alpine-tomcat]$ cat > index.html <<'EOF'
<!DOCTYPE html>
<html>
<head>
    <title>Alpine Tomcat Docker Lab</title>
</head>
<body>
    <h1>Task 3 Completed Successfully!</h1>
    <p>Tomcat is running inside a custom Alpine Linux Docker image.</p>
    <p>Platform: AWS EC2 - Amazon Linux</p>
</body>
</html>
EOF
[ec2-user@ip-172-31-30-255 task3-alpine-tomcat]$ cat index.html
<!DOCTYPE html>
<html>
<head>
    <title>Alpine Tomcat Docker Lab</title>
</head>
<body>
    <h1>Task 3 Completed Successfully!</h1>
    <p>Tomcat is running inside a custom Alpine Linux Docker image.</p>
    <p>Platform: AWS EC2 - Amazon Linux</p>
</body>
</html>
[ec2-user@ip-172-31-30-255 task3-alpine-tomcat]$ cat > Dockerfile <<'EOF'
FROM alpine:3.22

LABEL maintainer="DevOps Lab"
LABEL purpose="Custom Alpine image with Apache Tomcat"

ARG TOMCAT_VERSION=10.1.61

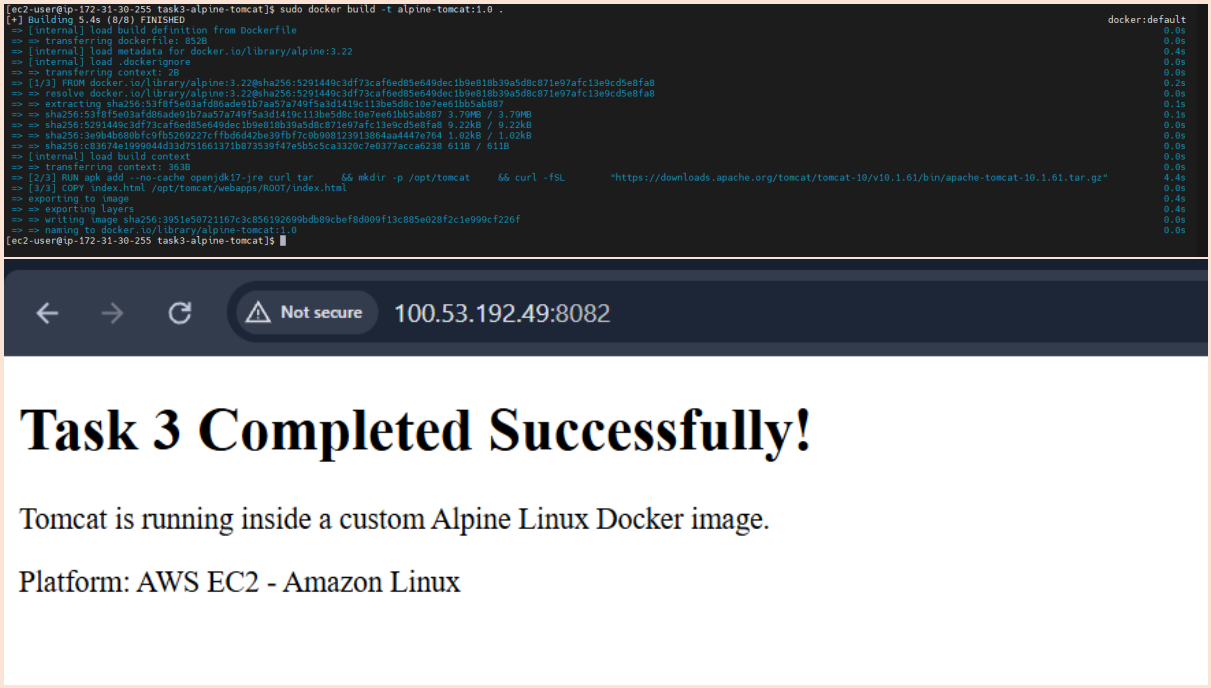
RUN apk add --no-cache openjdk17-jre curl tar \
    && mkdir -p /opt/tomcat \
    && curl -fSL \
       "https://downloads.apache.org/tomcat/tomcat-10/v${TOMCAT_VERSION}/bin/apache-tomcat-${TOMCAT_VERSION}.tar.gz" \
       -o /tmp/tomcat.tar.gz \
    && tar -xzf /tmp/tomcat.tar.gz \
       -C /opt/tomcat --strip-components=1 \
    && rm /tmp/tomcat.tar.gz \
    && apk del curl tar \
    && addgroup -S tomcat \
    && adduser -S -D -H -G tomcat tomcat \
    && chown -R tomcat:tomcat /opt/tomcat

COPY index.html /opt/tomcat/webapps/ROOT/index.html

EXPOSE 8080

USER tomcat

CMD ["/opt/tomcat/bin/catalina.sh", "run"]
```



7. Conclusion

Successfully built a custom Alpine Linux Docker image with Java and Apache Tomcat and deployed a sample web page inside a running container. This exercise demonstrated Dockerfile-based image creation, port mapping, container startup, and application validation.

Task 4: Create single stage and multi stage docker file using this source code:
https://github.com/betawins/multi-stage-example.git

1. Objective
To create and compare single-stage and multi-stage Docker images for a Spring Boot application using the multi-stage-example GitHub repository.

2. Purpose
Docker images can include compilers, build tools, source code, and other files that are not needed when the application runs. Multi-stage builds separate the build environment from the runtime environment, helping reduce the final image size and attack surface.

3. Implementation

Step 1: Clone the repository
Repository: https://github.com/betawins/multi-stage-example.git
Clone the repository and move into the project directory.

Step 2: Create the .dockerignore file
Exclude unnecessary files and directories from the Docker build context.

```
cat > .dockerignore <<'EOF'
.git
target
*.log
EOF
```

Step 3: Create the single-stage Dockerfile
Create a file named Dockerfile.single.

```
FROM maven:3.9.9-eclipse-temurin-8

WORKDIR /app

COPY pom.xml .
COPY .mvn .mvn
COPY mvnw .
COPY src ./src

RUN mvn -B clean package

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "/app/target/multi-stage-example-0.0.1-SNAPSHOT.jar"]
```

Build the image:

```
docker build -f Dockerfile.single -t multi-stage-single:v1 .
```

Step 4: Create the multi-stage Dockerfile
Create a file named Dockerfile.multi.

```
FROM maven:3.9.9-eclipse-temurin-8 AS builder

WORKDIR /app

COPY pom.xml .
COPY .mvn .mvn
COPY mvnw .
COPY src ./src

RUN mvn -B clean package

FROM eclipse-temurin:8-jre-jammy AS runtime

WORKDIR /app

COPY --from=builder /app/target/multi-stage-example-0.0.1-SNAPSHOT.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

Build the image:

```
docker build -f Dockerfile.multi -t multi-stage-app:v1 .
```

Step 5: Compare image sizes

docker images

docker image inspect multi-stage-single:v1 --format='Single-stage image size: {{.Size}} bytes'

docker image inspect multi-stage-app:v1 --format='Multi-stage image size: {{.Size}} bytes'

Step 6: Run the containers

docker run -d --name task4-single -p 8083:8080 multi-stage-single:v1

docker run -d --name task4-multi -p 8084:8080 multi-stage-app:v1

Validate the containers:

docker ps

docker logs task4-single

docker logs task4-multi

The application may return HTTP 404 at / if no root endpoint is configured. Check application startup logs and configured application endpoints before concluding that the container failed.

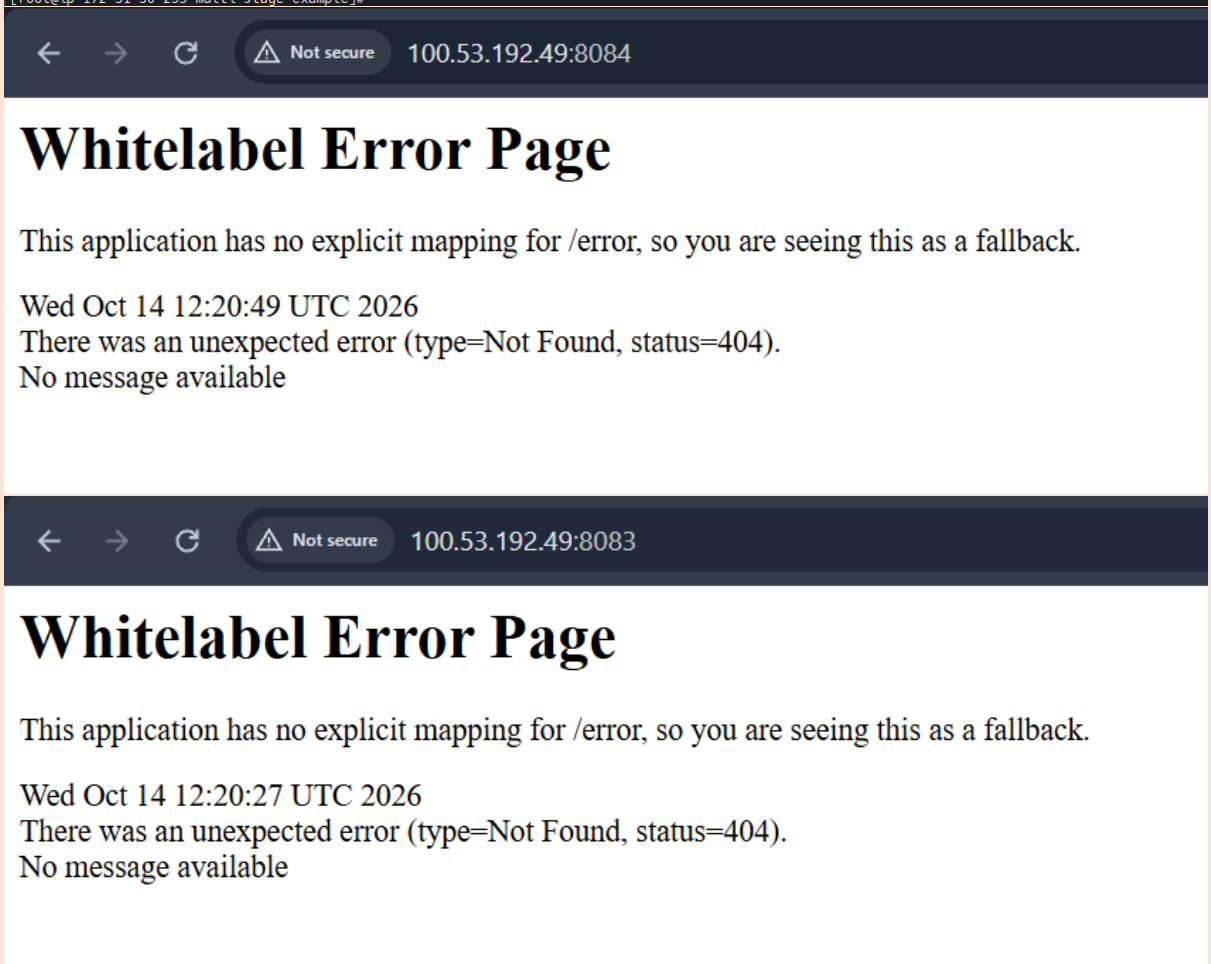
5. Key Differences

Single-stage build	**Multi-stage build**
Uses one build/runtime image stage	Uses separate builder and runtime stages
May retain Maven and build tools	Final image excludes the Maven build environment
Often has a larger final image	Often has a smaller final image
Simpler for small projects	Better suited to production image optimization

6.Implementation



```
[root@ip-172-31-30-255 multi-stage-example]# docker build -f Dockerfile.single -t multi-stage-single:v1 .
[+] Building 28.7s (12/12) FINISHED
 => [internal] load build definition from Dockerfile.single
 => => transferring dockerfile: 335B
 => [internal] load metadata for docker.io/library/maven:3.9.9-eclipse-temurin-8
 => [internal] load .dockerignore
 => => transferring context: 118B
 => [1/7] FROM docker.io/library/maven:3.9.9-eclipse-temurin-8@sha256:e20e4fc05fb5480c884a1f750c362ed0e91aca15cec5736e12796713710ec9b3
 => => resolve docker.io/library/maven:3.9.9-eclipse-temurin-8@sha256:e20e4fc05fb5480c884a1f750c362ed0e91aca15cec5736e12796713710ec9b3
 => => sha256:e20e4fc05fb5480c884a1f750c362ed0e91aca15cec5736e12796713710ec9b3 5.27kB / 5.27kB
 => => sha256:4977e75de990c5923bffd540c03c76a91075d07bddd8a42d71126e752a2bd34d 2.89kB / 2.89kB
 => => sha256:b228cc94c6d3b34f0a65e50876fa09b24471c822980694f9f6b1d6047c1f02d5 9.06kB / 9.06kB
 => => sha256:d9d352c11bbd3880007953ed6eec1cbace76898828f3434984a0ca60672fdf5a 29.72MB / 29.72MB
 => => sha256:e736d88fb0a15991a9783cf111345608b20d8c5b4a8245db0c577b88085725bc 54.72MB / 54.72MB
 => => sha256:ced9f6a6a78db418463f4b4676ae7de40c0f6ae0b04f795c13869a4e3c839a9c 16.97MB / 16.97MB
 => => sha256:db3bed063255930525fa83fb9863b101a6f85643272d11bbc3eb0fd8ba793047 126B / 126B
 => => extracting sha256:d9d352c11bbd3880007953ed6eec1cbace76898828f3434984a0ca60672fdf5a
 => => sha256:fde0f084c0fbafc0988e11d8a81a0e5caf2b1a2d7723862aa2a8e3e227830585 2.31kB / 2.31kB
 => => sha256:3e6d6db1241c7caa935dadb7f853bd4c3155e60a780adcb9bdc5edc8508c393d 22.53MB / 22.53MB
 => => sha256:04ecbf5554935c2dc61b09dc10e1a42694f00f16cd88fd200ba8fd7a310ba014 9.17MB / 9.17MB
 => => sha256:afc38625ccffd7d8ffb6fe14fc0672b56e56251627be4ba65b87e53c8f016cfe 849B / 849B
 => => sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb5577484a6d75e68dc38e8acc1 32B / 32B
 => => sha256:be23d9ab32c633dc51f033a5bf6eaae4c09a8fc93ef730d2eba4f1e5d81da29b 154B / 154B
 => => extracting sha256:ced9f6a6a78db418463f4b4676ae7de40c0f6ae0b04f795c13869a4e3c839a9c
 => => extracting sha256:e736d88fb0a15991a9783cf111345608b20d8c5b4a8245db0c577b88085725bc
 => => extracting sha256:db3bed063255930525fa83fb9863b101a6f85643272d11bbc3eb0fd8ba793047
 => => extracting sha256:fde0f084c0fbafc0988e11d8a81a0e5caf2b1a2d7723862aa2a8e3e227830585
 => => extracting sha256:3e6d6db1241c7caa935dadb7f853bd4c3155e60a780adcb9bdc5edc8508c393d
 => => extracting sha256:04ecbf5554935c2dc61b09dc10e1a42694f00f16cd88fd200ba8fd7a310ba014
 => => extracting sha256:afc38625ccffd7d8ffb6fe14fc0672b56e56251627be4ba65b87e53c8f016cfe
 => => extracting sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb5577484a6d75e68dc38e8acc1
 => => extracting sha256:be23d9ab32c633dc51f033a5bf6eaae4c09a8fc93ef730d2eba4f1e5d81da29b
 => [internal] load build context
 => => transferring context: 70.58kB
 => [2/7] WORKDIR /app
 => [3/7] COPY pom.xml .
 => [4/7] COPY .mvn .mvn
 => [5/7] COPY mvnw .
 => [6/7] COPY src ./src
 => [7/7] RUN mvn -B clean package
 => exporting to image
 => => exporting layers
 => => writing image sha256:77784e8bdc57925bc82531d6488399340ff6000b0adf4989d51377df9241779a
 => => naming to docker.io/library/multi-stage-single:v1
[root@ip-172-31-30-255 multi-stage-example]#
```



7. Conclusion

The task demonstrates how to build and run a Java application using two Docker build strategies. Multi-stage builds help keep production images leaner by copying the packaged application artifact into a separate runtime image.

Task 5: Install Docker Compose and Execute a Sample Application

1. Objective

To install Docker Compose on an Amazon Linux 2023 EC2 instance and deploy containerized applications using a Compose configuration file. This task covers a basic Nginx and Redis application and a WordPress application connected to a MySQL database.

2. Purpose

Docker Compose allows multiple containers to be defined and managed together using a YAML configuration file.

In this lab, Docker Compose is used to:

- Run multiple services with a single command.
- Configure container networking and port mapping.
- Use environment variables for application configuration.
- Persist data using named volumes.
- Manage application startup, status, logs, and restarts.

3. Architecture

Application 1: Nginx and Redis

User / Browser
    |
    | HTTP :8085
    v
Nginx Container
    |
    | Docker Compose network
    v
Redis Container

The original sample runs Nginx and Redis as separate containers. Redis is not exposed directly to the host.

Application 2: WordPress and MySQL

User / Browser
    |
    | HTTP :8086
    v
WordPress Container
    |
    | MySQL protocol :3306
    v
MySQL Container
    |
    v
Persistent Database Volume

WordPress connects to MySQL through the internal Compose network using the service name db. WordPress files and database data are stored in separate named volumes.

4. Implementation

Part A: Install Docker Compose

Step 1: Verify Docker

```
docker --version
docker info
```

If Docker is not running, check its service: systemctl status docker

Step 2: Install the Docker Compose standalone CLI plugin

The package was not available through the configured DNF repositories, so the Compose binary was installed directly.

```
mkdir -p /usr/local/lib/docker/cli-plugins

curl -fL \
  https://github.com/docker/compose/releases/download/v5.6.0/docker-compose-linux-x86_64 \
  -o /usr/local/lib/docker/cli-plugins/docker-compose

chmod +x /usr/local/lib/docker/cli-plugins/docker-compose
```

Verify the installation: `docker compose version`

Expected version in this lab: Docker Compose version v5.6.0

Part B: Deploy Nginx and Redis

Step 1: Create the project directory

```
mkdir -p /root/task5-compose
cd /root/task5-compose
```

Step 2: Create the Compose file: vi compose.yaml
Configuration:

```
services:
  web:
    image: nginx:alpine
    ports:
      - "8085:80"
    restart: unless-stopped

  redis:
    image: redis:alpine
    restart: unless-stopped
```

Save the file.

Step 3: Start the application

```
docker compose up -d
```

This command downloads the required images if they are not already present, creates the containers, and starts the services in the background.

Step 4: Validate the services

docker compose ps

Test the Nginx web server locally: curl -I http://localhost:8085

Expected HTTP response: HTTP/1.1 200 OK

Redis is available to containers on the Compose network but is not published on a host port in this configuration.

Part C: Deploy WordPress and MySQL

Step 1: Create a separate project directory

The WordPress deployment uses a separate directory so that the original Nginx and Redis project remains intact.

mkdir -p /root/task5-wordpress
cd /root/task5-wordpress

Step 2: Generate database passwords

openssl rand -hex 24
openssl rand -hex 24

Use the first output for the MySQL root password and the second for the WordPress database user's password. Store these securely.

Step 3: Create the environment file

vi .env

Configuration:

MYSQL_ROOT_PASSWORD=REPLACE_WITH_FIRST_PASSWORD
MYSQL_PASSWORD=REPLACE_WITH_SECOND_PASSWORD
MYSQL_DATABASE=wordpress
MYSQL_USER=wpuser
Replace both placeholders with the generated passwords.

Restrict access to the file: chmod 600 .env

Do not commit this file to a public Git repository or share its contents.

Step 4: Create the WordPress Compose file

vi compose.yaml

Configuration:

```
services:
  db:
    image: mysql:8.0
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD:?Set MYSQL_ROOT_PASSWORD in .env}
```

```
    MYSQL_DATABASE: ${MYSQL_DATABASE:-wordpress}
    MYSQL_USER: ${MYSQL_USER:-wpuser}
    MYSQL_PASSWORD: ${MYSQL_PASSWORD:?Set MYSQL_PASSWORD in .env}
  volumes:
    - wordpress_db_data:/var/lib/mysql
  networks:
    - wordpress-network

 wordpress:
  image: wordpress:latest
  restart: unless-stopped
  depends_on:
    - db
  ports:
    - "8086:80"
  environment:
    WORDPRESS_DB_HOST: db:3306
    WORDPRESS_DB_NAME: ${MYSQL_DATABASE:-wordpress}
    WORDPRESS_DB_USER: ${MYSQL_USER:-wpuser}
    WORDPRESS_DB_PASSWORD: ${MYSQL_PASSWORD:?Set MYSQL_PASSWORD in .env}
  volumes:
    - wordpress_files:/var/www/html
  networks:
    - wordpress-network

volumes:
 wordpress_db_data:
 wordpress_files:

networks:
 wordpress-network:
```

Important port detail: The original Nginx application already uses host port 8085. WordPress therefore uses host port 8086, mapped to container port 80.

Step 5: Validate the Compose configuration

```
cd /root/task5-wordpress
docker compose config
```

If the configuration renders without errors, start the services: `docker compose up -d`

The images used are:

- wordpress:latest
- mysql:8.0

Docker Compose creates the project network, named volumes, and containers.

Step 6: Check container status

docker compose ps

Both the db and wordpress services should be running. The database may need time to initialize before WordPress can connect successfully.

Check logs if necessary:

docker compose logs --tail=100 db
docker compose logs --tail=100 wordpress

Step 7: Open the WordPress installation page

Find the EC2 instance's current public IPv4 address in the AWS console and open this URL:
http://YOUR_EC2_PUBLIC_IP:8086

Step 8: Validate persistence and restart behavior

Check the services: docker compose ps

Restart the WordPress project: docker compose restart

Refresh the browser and verify that the site remains available.

The named volumes retain WordPress files and database data across normal container restarts and recreation. Do not remove these volumes during the test.

5. Configuration Details

Configuration	Purpose
compose.yaml	Defines services, networks, volumes, ports, and environment variables
docker compose up -d	Starts services in detached mode
docker compose ps	Displays service and container status
docker compose logs	Displays application and database logs
docker compose restart	Restarts the project's services
8085:80	Maps host port 8085 to Nginx container port 80
8086:80	Maps host port 8086 to WordPress container port 80
db:3306	WordPress's internal address for the MySQL service
wordpress_db_data	Persists MySQL database files
wordpress_files	Persists WordPress application files
.env	Stores database configuration and passwords outside the Compose YAML

Networking

The WordPress and MySQL containers communicate through the wordpress-network network. The database does not need a public port mapping because WordPress accesses it internally.

Persistent storage

The database uses the wordpress_db_data volume, mounted at /var/lib/mysql. WordPress uses wordpress_files, mounted at /var/www/html.

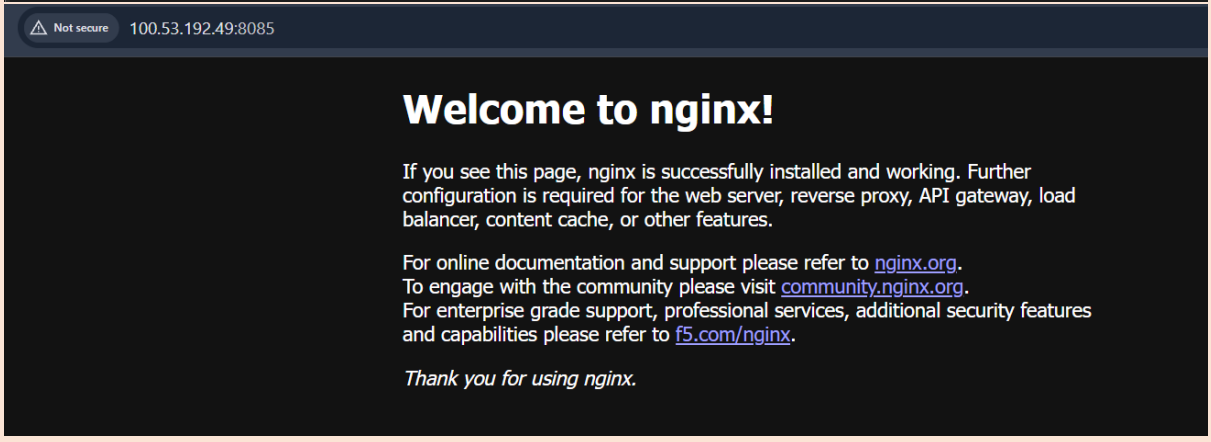
Named volumes are important because a container's writable layer is not intended to be the sole storage location for data that must survive container replacement.

Also verify the EC2 security group's inbound rule for TCP port 8086 and confirm the correct public IPv4 address.

6.Implementation

```
[root@ip-172-31-30-255 multi-stage-example]# mkdir -p /usr/local/lib/docker/cli-plugins
[root@ip-172-31-30-255 multi-stage-example]# curl -fL https://github.com/docker/compose/releases/download/v5.6.0/docker-compose-linux-x86_64 \
  -o /usr/local/lib/docker/cli-plugins/docker-compose
  % Total    % Received % Xferd  Average Speed  Time    Time    Time   Current
                                 Dload  Upload  Total   Spent   Left   Speed
  0      0   0      0   0      0      0      0                              0
100 31.14M 100 31.14M   0      0 216.4M      0                              0
[root@ip-172-31-30-255 multi-stage-example]# chmod +x /usr/local/lib/docker/cli-plugins/docker-compose
[root@ip-172-31-30-255 multi-stage-example]# docker compose version
Docker Compose version v5.6.0
[root@ip-172-31-30-255 multi-stage-example]# docker compose ls
NAME                STATUS              CONFIG FILES
[root@ip-172-31-30-255 multi-stage-example]# mkdir -p /root/task5-compose
cd /root/task5-compose
[root@ip-172-31-30-255 task5-compose]# cat > compose.yaml <<'EOF'
services:
  web:
    image: nginx:alpine
    ports:
      - "8085:80"
    restart: unless-stopped

  redis:
    image: redis:alpine
    restart: unless-stopped
EOF
[root@ip-172-31-30-255 task5-compose]# docker compose up -d
[+] up 11/11
 ✔ Image redis:alpine              Pulled
 ✔ Network task5-compose_default   Created
 ✔ Container task5-compose-web-1   Started
 ✔ Container task5-compose-redis-1 Started
[root@ip-172-31-30-255 task5-compose]#
```

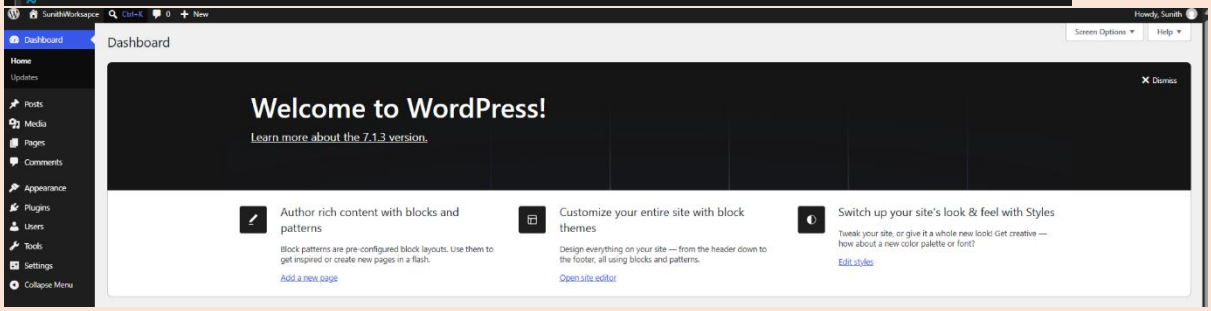


```yaml
services:
  db:
    image: mysql:8.0
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD:?Set MYSQL_ROOT_PASSWORD in .env}
      MYSQL_DATABASE: ${MYSQL_DATABASE:-wordpress}
      MYSQL_USER: ${MYSQL_USER:-wpuser}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD:?Set MYSQL_PASSWORD in .env}
    volumes:
      - wordpress_db_data:/var/lib/mysql
    networks:
      - wordpress-network

  wordpress:
    image: wordpress:latest
    restart: unless-stopped
    depends_on:
      - db
    ports:
      - "8085:80"
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_NAME: ${MYSQL_DATABASE:-wordpress}
      WORDPRESS_DB_USER: ${MYSQL_USER:-wpuser}
      WORDPRESS_DB_PASSWORD: ${MYSQL_PASSWORD:?Set MYSQL_PASSWORD in .env}
    volumes:
      - wordpress_files:/var/www/html
    networks:
      - wordpress-network

volumes:
  wordpress_db_data:
  wordpress_files:

networks:
  wordpress-network:
```



6. Conclusion

Task 5 demonstrates the installation and use of Docker Compose on Amazon Linux 2023. The original Nginx and Redis sample demonstrates basic multi-container deployment, while WordPress and MySQL demonstrate an application-and-database architecture with internal networking, environment-based configuration, host port mapping, and persistent storage.

The WordPress application was successfully started and its installation page opened in the browser. The lab resources remain available for the remaining tasks and will be cleaned up together at the end.

Clarify the Nginx–Redis connectionAdd the missing Docker Compose prerequisites

Task 6: Implement solution to scan images when pushed to docker registry.

Scan Docker Images When Pushing to a Docker Registry

1. Objective

To scan Docker images for security vulnerabilities using Trivy and publish a test image to Docker Hub.

2. Purpose

Docker images contain an operating system base, libraries, application dependencies, and application files. Some of these components may contain known security vulnerabilities.

Image scanning helps identify vulnerable packages before an image is used in a deployment. Integrating scanning into a container publishing workflow helps teams make more informed

3. Implementation

Step 1: Install Trivy

Configure the Trivy RPM repository and install the scanner.

```
cat > /etc/yum.repos.d/trivy.repo <<'EOF'
[trivy]
name=Trivy repository
baseurl=https://aquasecurity.github.io/trivy-repo/rpm/releases/$basearch/
gpgcheck=1
enabled=1
gpgkey=https://aquasecurity.github.io/trivy-repo/rpm/public.key
EOF
dnf install -y trivy
trivy --version
```

Observed version: Trivy 0.76.0.

Step 2: Authenticate to Docker Hub

docker login

Step 3: Scan the base image

```
trivy image nginx:alpine
```

The first scan downloaded the vulnerability database and inspected the Alpine Linux packages contained in the image.
For a concise report showing only serious vulnerabilities, run:

```
trivy image --severity HIGH,CRITICAL nginx:alpine
```

Step 4: Create a dedicated image

Create a lab directory:

```
mkdir -p /root/task6-registry-scan
cd /root/task6-registry-scan
```

Create a custom index.html page and a Dockerfile based on nginx:alpine.

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

Step 5: Build the image

docker build -t sunith18/docker-registry-scan-lab:v1 .

The image uses the Docker Hub account name as the namespace and v1 as the version tag.

Step 6: Scan the custom image

trivy image --severity HIGH,CRITICAL sunith18/docker-registry-scan-lab:v1

Review the affected packages, vulnerability severity, installed versions, and available fixed versions before publishing the image.

Step 7: Push the image to Docker Hub

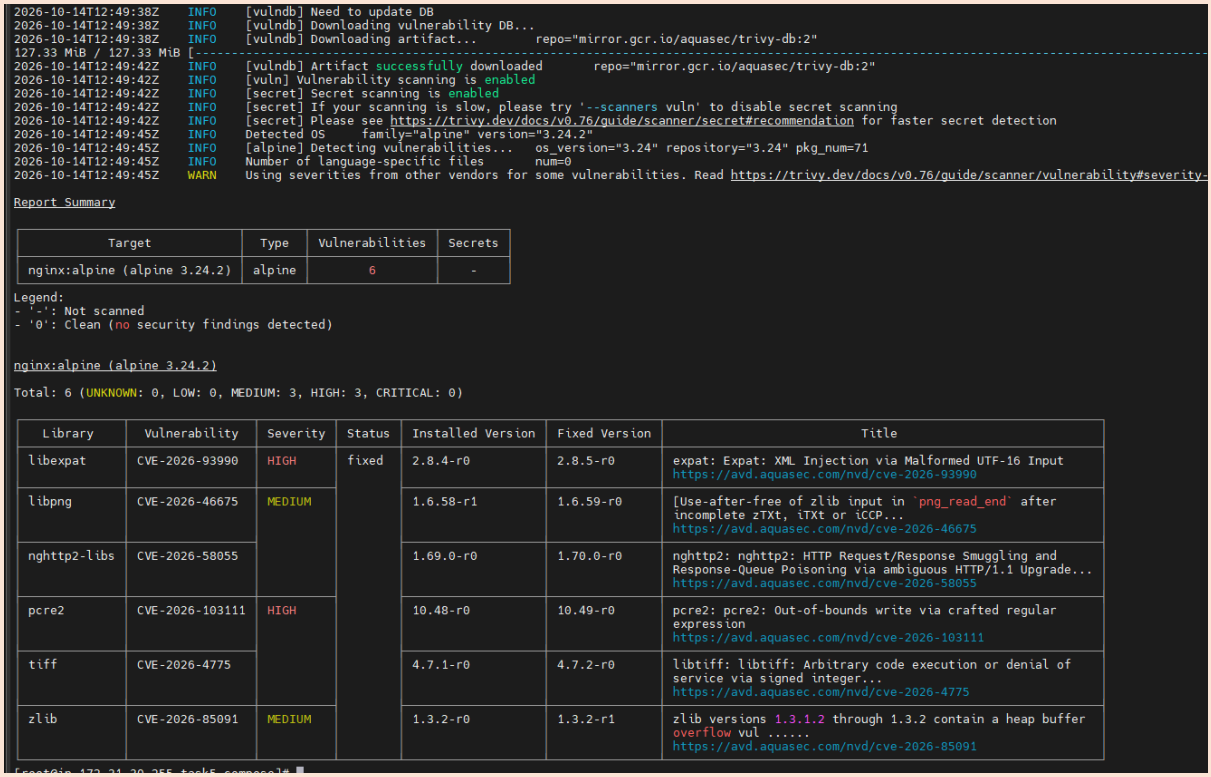
docker push sunith18/docker-registry-scan-lab:v1

Step 8: Scan the published image

trivy image --severity HIGH,CRITICAL docker.io/sunith18/docker-registry-scan-lab:v1

This verifies that the published image can be referenced and assessed using Trivy.

4.Implementation



```
2026-10-14T12:49:38Z    INFO    [vulndb] Need to update DB
2026-10-14T12:49:38Z    INFO    [vulndb] Downloading vulnerability DB...
2026-10-14T12:49:38Z    INFO    [vulndb] Downloading artifact...        repo="mirror.gcr.io/aquasec/trivy-db:2"
127.33 MiB / 127.33 MiB [-----------------------------------------------------------------------------------------------------------------------]
2026-10-14T12:49:42Z    INFO    [vulndb] Artifact successfully downloaded       repo="mirror.gcr.io/aquasec/trivy-db:2"
2026-10-14T12:49:42Z    INFO    [vuln] Vulnerability scanning is enabled
2026-10-14T12:49:42Z    INFO    [secret] Secret scanning is enabled
2026-10-14T12:49:42Z    INFO    [secret] If your scanning is slow, please try '--scanners vuln' to disable secret scanning
2026-10-14T12:49:42Z    INFO    [secret] Please see https://trivy.dev/docs/v0.76/guide/scanner/secret#recommendation for faster secret detection
2026-10-14T12:49:45Z    INFO    Detected OS     family="alpine" version="3.24.2"
2026-10-14T12:49:45Z    INFO    [alpine] Detecting vulnerabilities...   os_version="3.24" repository="3.24" pkg_num=71
2026-10-14T12:49:45Z    INFO    Number of language-specific files       num=0
2026-10-14T12:49:45Z    WARN    Using severities from other vendors for some vulnerabilities. Read https://trivy.dev/docs/v0.76/guide/scanner/vulnerability#severity-

Report Summary

┌──────────────────────────┬────────┬─────────────────┬─────────┐
│          Target          │  Type  │ Vulnerabilities │ Secrets │
├──────────────────────────┼────────┼─────────────────┼─────────┤
│ nginx:alpine (alpine 3.24.2) │ alpine │        6        │    -    │
└──────────────────────────┴────────┴─────────────────┴─────────┘

Legend:
- '-': Not scanned
- '0': Clean (no security findings detected)


nginx:alpine (alpine 3.24.2)

Total: 6 (UNKNOWN: 0, LOW: 0, MEDIUM: 3, HIGH: 3, CRITICAL: 0)

┌──────────┬────────────────┬──────────┬────────┬───────────────────┬───────────────┬──────────────────────────────────────────────────────────────┐
│ Library  │ Vulnerability  │ Severity │ Status │ Installed Version │ Fixed Version │                            Title                             │
├──────────┼────────────────┼──────────┼────────┼───────────────────┼───────────────┼──────────────────────────────────────────────────────────────┤
│ libexpat │ CVE-2026-93990 │ HIGH     │ fixed  │ 2.8.4-r0          │ 2.8.5-r0      │ expat: Expat: XML Injection via Malformed UTF-16 Input       │
│          │                │          │        │                   │               │ https://avd.aquasec.com/nvd/cve-2026-93990                   │
├──────────┼────────────────┼──────────┤        │                   │               ├──────────────────────────────────────────────────────────────┤
│ libpng   │ CVE-2026-46675 │ MEDIUM   │        │ 1.6.58-r1         │ 1.6.59-r0     │ [Use-after-free of zlib input in `png_read_end` after        │
│          │                │          │        │                   │               │ incomplete zTXt, iTXt or iCCP...                             │
│          │                │          │        │                   │               │ https://avd.aquasec.com/nvd/cve-2026-46675                   │
├──────────┼────────────────┼──────────┤        │                   │               ├──────────────────────────────────────────────────────────────┤
│ nghttp2-libs │ CVE-2026-58055 │      │        │ 1.69.0-r0         │ 1.70.0-r0     │ nghttp2: nghttp2: HTTP Request/Response Smuggling and        │
│          │                │          │        │                   │               │ Response-Queue Poisoning via ambiguous HTTP/1.1 Upgrade...   │
│          │                │          │        │                   │               │ https://avd.aquasec.com/nvd/cve-2026-58055                   │
├──────────┼────────────────┼──────────┼────────┼───────────────────┼───────────────┼──────────────────────────────────────────────────────────────┤
│ pcre2    │ CVE-2026-103111 │ HIGH    │        │ 10.48-r0          │ 10.49-r0      │ pcre2: pcre2: Out-of-bounds write via crafted regular        │
│          │                │          │        │                   │               │ expression                                                   │
│          │                │          │        │                   │               │ https://avd.aquasec.com/nvd/cve-2026-103111                  │
├──────────┼────────────────┼──────────┤        │                   │               ├──────────────────────────────────────────────────────────────┤
│ tiff     │ CVE-2026-4775  │          │        │ 4.7.1-r0          │ 4.7.2-r0      │ libtiff: libtiff: Arbitrary code execution or denial of      │
│          │                │          │        │                   │               │ service via signed integer...                                │
│          │                │          │        │                   │               │ https://avd.aquasec.com/nvd/cve-2026-4775                    │
├──────────┼────────────────┼──────────┼────────┼───────────────────┼───────────────┼──────────────────────────────────────────────────────────────┤
│ zlib     │ CVE-2026-85091 │ MEDIUM   │        │ 1.3.2-r0          │ 1.3.2-r1      │ zlib versions 1.3.1.2 through 1.3.2 contain a heap buffer    │
│          │                │          │        │                   │               │ overflow vul ......                                          │
│          │                │          │        │                   │               │ https://avd.aquasec.com/nvd/cve-2026-85091                   │
└──────────┴────────────────┴──────────┴────────┴───────────────────┴───────────────┴──────────────────────────────────────────────────────────────┘
```

```
[root@ip-172-31-30-255 task6-registry-scan]# docker build -t sunith18/docker-registry-scan-lab:v1 .
[+] Building 0.2s (7/7) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 174B
 => [internal] load metadata for docker.io/library/nginx:alpine
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [internal] load build context
 => => transferring context: 294B
 => [1/2] FROM docker.io/library/nginx:alpine
 => [2/2] COPY index.html /usr/share/nginx/html/index.html
 => exporting to image
 => => exporting layers
 => => writing image sha256:47e01db3e1483586d125449d43e3964d5e09fbf6e99176248e682d29cd515be1
 => => naming to docker.io/sunith18/docker-registry-scan-lab:v1
[root@ip-172-31-30-255 task6-registry-scan]# trivy image --severity HIGH,CRITICAL sunith18/docker-registry-scan-lab:v1
2026-10-14T12:53:01Z    INFO    [vuln] Vulnerability scanning is enabled
2026-10-14T12:53:01Z    INFO    [secret] Secret scanning is enabled
2026-10-14T12:53:01Z    INFO    [secret] If your scanning is slow, please try '--scanners vuln' to disable secret scanning
2026-10-14T12:53:01Z    INFO    [secret] Please see https://trivy.dev/docs/v0.76/guide/scanner/secret#recommendation for faster secret detection
2026-10-14T12:53:01Z    INFO    Detected OS     family="alpine" version="3.24.2"
2026-10-14T12:53:01Z    INFO    [alpine] Detecting vulnerabilities...   os_version="3.24" repository="3.24" pkg_num=71
2026-10-14T12:53:01Z    INFO    Number of language-specific files       num=0
2026-10-14T12:53:01Z    WARN    Using severities from other vendors for some vulnerabilities. Read https://trivy.dev/docs/v0.76/guide/scanner/vulnerability#severity-selection for details.

Report Summary

┌──────────────────────────────────────────────┬────────┬─────────────────┬─────────┐
│                    Target                    │  Type  │ Vulnerabilities │ Secrets │
├──────────────────────────────────────────────┼────────┼─────────────────┼─────────┤
│ sunith18/docker-registry-scan-lab:v1 (alpine 3.24.2) │ alpine │        3        │    -    │
└──────────────────────────────────────────────┴────────┴─────────────────┴─────────┘
Legend:
- '-': Not scanned
- '0': Clean (no security findings detected)


sunith18/docker-registry-scan-lab:v1 (alpine 3.24.2)

Total: 3 (HIGH: 3, CRITICAL: 0)

┌─────────┬────────────────┬──────────┬────────┬───────────────────┬───────────────┬──────────────────────────────────────────────────────────────┐
│ Library │ Vulnerability  │ Severity │ Status │ Installed Version │ Fixed Version │                            Title                             │
├─────────┼────────────────┼──────────┼────────┼───────────────────┼───────────────┼──────────────────────────────────────────────────────────────┤
│ libexpat │ CVE-2026-93990 │ HIGH     │ fixed  │ 2.8.4-r0          │ 2.8.5-r0      │ expat: Expat: XML Injection via Malformed UTF-16 Input       │
│         │                │          │        │                   │               │ https://avd.aquasec.com/nvd/cve-2026-93990                   │
├─────────┼────────────────┼──────────┼────────┼───────────────────┼───────────────┼──────────────────────────────────────────────────────────────┤
│ pcre2   │ CVE-2026-103111 │          │        │ 10.48-r0          │ 10.49-r0      │ pcre2: pcre2: Out-of-bounds write via crafted regular        │
│         │                │          │        │                   │               │ expression                                                   │
└─────────┴────────────────┴──────────┴────────┴───────────────────┴───────────────┴──────────────────────────────────────────────────────────────┘
```

```
[root@ip-172-31-30-255 task6-registry-scan]# docker push sunith18/docker-registry-scan-lab:v1
The push refers to repository [docker.io/sunith18/docker-registry-scan-lab]
66eb1b72095d: Pushed
001b5608a929: Mounted from library/nginx
c86f6810da0b: Mounted from library/nginx
41473198c04a: Mounted from library/nginx
149ab1e5d6ca: Mounted from library/nginx
2a7ed9afc1c3: Mounted from library/nginx
641b74319bab: Mounted from library/nginx
379f51bf5b58: Mounted from library/nginx
74d97c428c51: Mounted from library/nginx
v1: digest: sha256:dc497dee3e1403ccce45978554a151fb8c8c0353a06fd19183e040d99133f40b size: 2196
[root@ip-172-31-30-255 task6-registry-scan]#
```

5. Conclusion

This task demonstrates container vulnerability scanning with Trivy and publishing an image to Docker Hub. The initial Nginx scan identified six vulnerabilities, illustrating why image scanning is an important part of a DevOps CI/CD workflow.

Task 7: Implement solution to scan images when pushed to AWS ECR.

Scan Docker Images When Pushed to Amazon ECR

1. Objective

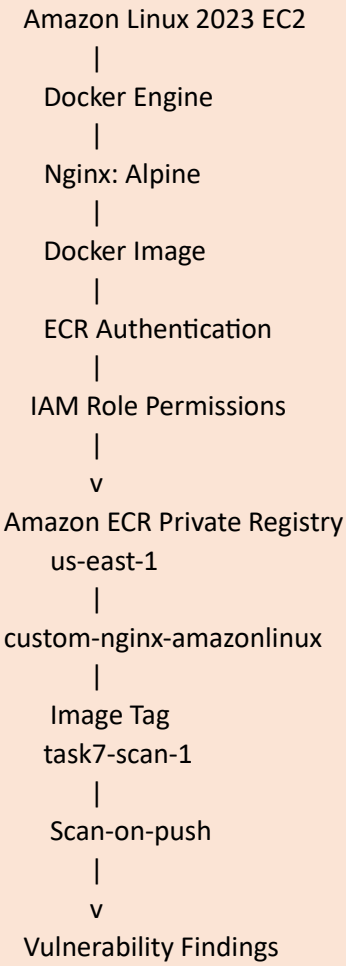
To configure Amazon Elastic Container Registry (ECR) image scanning, push a Docker image to a private ECR repository, and inspect the vulnerability findings.

2. Purpose

Amazon ECR stores and manages container images used in deployments. Image scanning helps identify known vulnerabilities in operating-system packages and other supported dependencies.

Enabling scan-on-push helps identify security issues when new images are published, before those images are promoted into deployment environments.

3. Architecture



4. Prerequisites

AWS account with access to Amazon ECR.

Amazon Linux 2023 EC2 instance.

Docker Engine and AWS CLI installed.

IAM role attached to the EC2 instance with appropriate ECR permissions.

Existing private ECR repository.

3. AWS Resources and IAM Configuration

AWS account details

IAM role used by EC2

Role name: EC2CustomNginxECRRole

The AWS CLI confirmed the assumed-role identity:

arn:aws:sts::567487919764:assumed-role/EC2CustomNginxECRRole/i-06c526df3f6981c03

The role is used by the EC2 instance to access ECR without storing long-term AWS access keys on the server.

IAM policy
Policy name: CustomNginxECRPushPolicy
Policy ARN: arn:aws:iam::567487919764:policy/CustomNginxECRPushPolicy

The existing policy was extended with the following permissions:

IAM action	Purpose
ecr:GetAuthorizationToken	Authenticate Docker to ECR
ecr:CreateRepository	Create a repository when authorized and required
ecr:DescribeRepositories	Inspect repository configuration
ecr:InitiateLayerUpload	Start uploading image layers
ecr:UploadLayerPart	Upload image-layer data
ecr:CompleteLayerUpload	Complete image-layer uploads
ecr:PutImage	Publish the image manifest
ecr:DescribeImages	Inspect published images
ecr:PutImageScanningConfiguration	Configure repository scan-on-push
ecr:DescribeImageScanFindings	Retrieve vulnerability findings
ecr:StartImageScan	Request a manual scan when supported by the configured scanning mode

Additional existing push/pull permissions were retained. Repository-specific actions were scoped to:

arn:aws:ecr:us-east-1:567487919764:repository/custom-nginx-amazonlinux

The ecr:GetAuthorizationToken permission uses Resource: "*" because ECR requires this authorization action to be granted at the account level.

4. Step-by-Step Implementation
Step 1: Verify AWS CLI

aws –version
aws sts get-caller-identity

The caller identity confirmed that the EC2 instance was using EC2CustomNginxECRRole in account 567487919764.

Step 2: Inspect the existing repository

```
aws ecr describe-repositories \
  --repository-names custom-nginx-amazonlinux \
  --region us-east-1
```

Initially, the repository reported:

```
{
  "scanOnPush": false
}
```

Step 3: Enable scan-on-push

After adding the required IAM permissions, run:

```
aws ecr put-image-scanning-configuration \
  --repository-name custom-nginx-amazonlinux \
  --image-scanning-configuration scanOnPush=true \
  --region us-east-1
```

Verify the configuration:

```
aws ecr describe-repositories \
  --repository-names custom-nginx-amazonlinux \
  --region us-east-1 \
  --query 'repositories[0].imageScanningConfiguration'
```

The repository should report scanOnPush as true.

Step 4: Authenticate Docker to ECR

```
aws ecr get-login-password --region us-east-1 \
  | docker login \
     --username AWS \
     --password-stdin 567487919764.dkr.ecr.us-east-1.amazonaws.com
```

This uses the EC2 instance role to obtain a temporary ECR login password.

Step 5: Pull the test image

```
docker pull nginx:alpine
```

Step 6: Tag the image for ECR

```
docker tag nginx:alpine \
  567487919764.dkr.ecr.us-east-1.amazonaws.com/custom-nginx-amazonlinux:task7-scan-1
```

A new tag was used so that the existing 1.0 image remained unchanged.

Step 7: Push the image

```
docker push \
  567487919764.dkr.ecr.us-east-1.amazonaws.com/custom-nginx-amazonlinux:task7-scan-1
```

Observed result: All image layers were pushed successfully.
Image digest: sha256:d9c56e383f4711392d7202ec08aa1ccebfc5d90dd41feebfecf42a8755f916bb

Step 8: Verify the published image

```
aws ecr describe-images \
  --repository-name custom-nginx-amazonlinux \
  --image-ids imageTag=task7-scan-1 \
  --region us-east-1
```

The response confirmed that the image existed and had status ACTIVE.

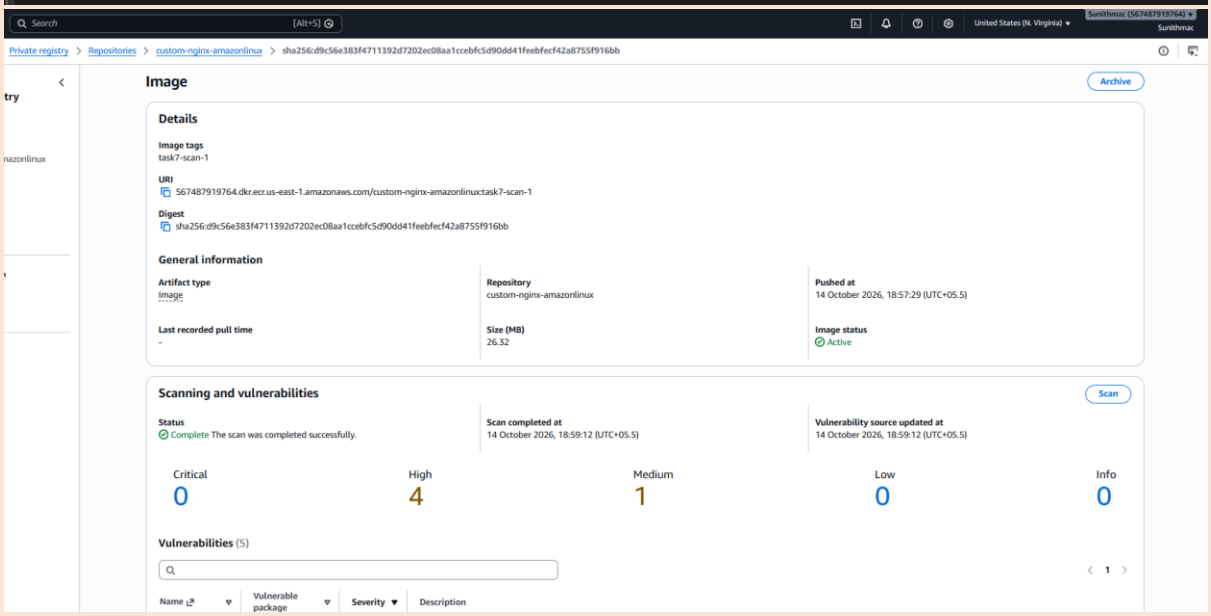
Step 9: Retrieve vulnerability findings

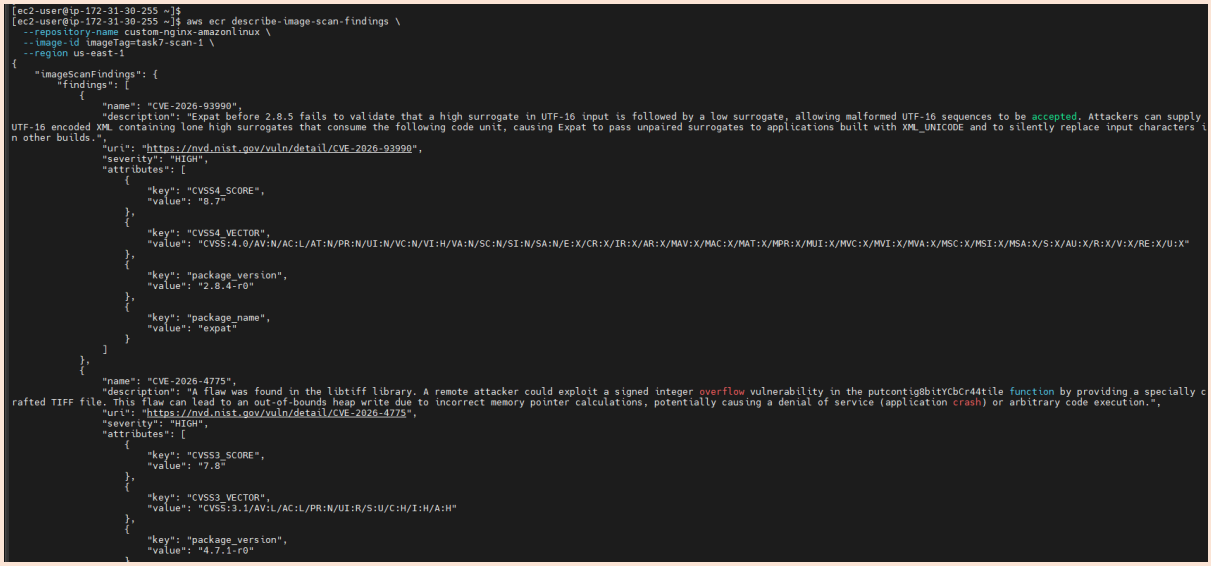
```
aws ecr describe-image-scan-findings \
  --repository-name custom-nginx-amazonlinux \
  --image-id imageTag=task7-scan-1 \
  --region us-east-1
```

The first request returned IN_PROGRESS. A subsequent request returned actual vulnerability findings.

5.Implementation

```
[ec2-user@ip-172-31-30-255 ~]$ aws ecr put-image-scanning-configuration \
  --repository-name custom-nginx-amazonlinux \
  --image-scanning-configuration scanOnPush=true \
  --region us-east-1
{
    "registryId": "567487919764",
    "repositoryName": "custom-nginx-amazonlinux",
    "imageScanningConfiguration": {
        "scanOnPush": true
    }
}
[ec2-user@ip-172-31-30-255 ~]$ aws ecr describe-repositories \
  --repository-names custom-nginx-amazonlinux \
  --region us-east-1 \
  --query 'repositories[0].imageScanningConfiguration'
{
    "scanOnPush": true
}
[ec2-user@ip-172-31-30-255 ~]$
```





6. Conclusion

The EC2 instance successfully authenticated to Amazon ECR through EC2CustomNginxECRRole, pushed the task7-scan-1 image to the existing private repository, and retrieved vulnerability findings. Scan-on-push was enabled, demonstrating how ECR scanning can be integrated into a container image publishing workflow.

Task 8: Create a Jenkins pipeline to create a docker image and push the image to Docker hub.

Jenkins Pipeline to Build, Scan, and Push a Docker Image to Docker Hub

1. Objective

To automate Docker image building, vulnerability scanning with Trivy, and publishing to Docker Hub using a Jenkins Declarative Pipeline.

2. Purpose

Manual Docker image building and publishing can be repetitive and inconsistent. Jenkins automates these operations so that the image follows a repeatable build and security-check process before being published.

3. Architecture

Git-independent sample application → Jenkins Pipeline → Docker Image Build → Trivy Vulnerability Scan → Docker Hub Authentication → Docker Hub Push.

4. Implementation

Step 1: Install and start Jenkins

Installed Java 21 and Jenkins on the EC2 server.

Step 2: Configure Docker access

Added the Jenkins service account to the Docker group and restarted Jenkins.

Verified Docker access using sudo -u jenkins docker version.

Step 3: Configure Trivy

Verified Trivy version 0.76.0. Used --config "" to prevent Trivy from loading an inaccessible configuration file from the working directory.

Configured the Trivy cache under /var/lib/jenkins/.cache/trivy.

Step 4: Configure Docker Hub credentials

Created a Jenkins username/password credential with ID dockerhub-creds. The password field contains a Docker Hub access token, which is injected into the pipeline through Jenkins Credentials Binding rather than hardcoded in the script.

Step 5: Create the pipeline

Created a Jenkins Pipeline job named: DockerHub_Pipeline.
The pipeline has four stages:

- Prepare Application — generates the HTML page and Dockerfile.
- Build Docker Image — builds sunith18/jenkins-docker-lab with the build-number tag and latest.
- Scan Image with Trivy — scans HIGH and CRITICAL vulnerabilities and fails the pipeline if CRITICAL findings are present.
- Push to Docker Hub — authenticates with Jenkins credentials and pushes both tags.

Step 6: Execute the pipeline

Ran build #1 and checked the Jenkins Console Output.

5. Configuration Details

Jenkins job: DockerHub_Pipeline
Build number: 1
Docker image: sunith18/jenkins-docker-lab
Tags: 1, latest
Image size: approximately 62.9 MB
Trivy severity threshold for reporting: HIGH, CRITICAL
Pipeline failure threshold: CRITICAL
Jenkins web port: 8090

6. Troubleshooting
Jenkins failed to start: Port 8080 was already in use. Moved Jenkins to port 8090.

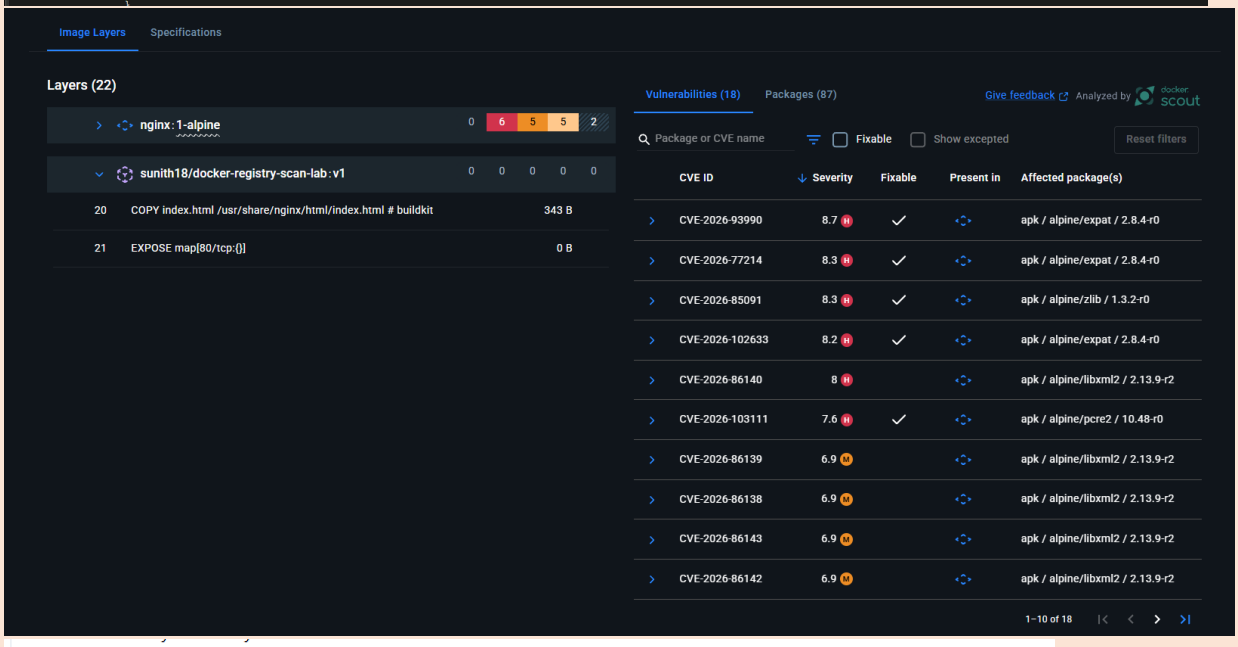
Docker permission denied: Added the Jenkins service account to the Docker group and restarted Jenkins.

Trivy configuration permission error: Disabled automatic configuration lookup with --config "".

Docker Hub authentication failure: Verify the Jenkins credential ID, username, and access-token permissions.
Trivy reports CRITICAL findings: Review the affected packages and update the base image before retrying.

7. Implementation:

```
[ec2-user@ip-172-31-30-255 ~]$
[ec2-user@ip-172-31-30-255 ~]$ aws ecr describe-image-scan-findings \
  --repository-name custom-nginx-amazonlinux \
  --image-id imageTag=task7-scan-1 \
  --region us-east-1
{
    "imageScanFindings": {
        "findings": [
            {
                "name": "CVE-2026-93990",
                "description": "Expat before 2.8.5 fails to validate that a high surrogate in UTF-16 input is followed by a low surrogate, allowing malformed UTF-16 sequences to be accepted. Attackers can supply
UTF-16 encoded XML containing lone high surrogates that consume the following code unit, causing Expat to pass unpaired surrogates to applications built with XML_UNICODE and to silently replace input characters i
n other builds.",
                "uri": "https://nvd.nist.gov/vuln/detail/CVE-2026-93990",
                "severity": "HIGH",
                "attributes": [
                    {
                        "key": "CVSS4_SCORE",
                        "value": "8.7"
                    },
                    {
                        "key": "CVSS4_VECTOR",
                        "value": "CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:H/VA:N/SC:N/SI:N/SA:N/E:X/CR:X/IR:X/AR:X/MAV:X/MAC:X/MAT:X/MPR:X/MUI:X/MVC:X/MVI:X/MVA:X/MSC:X/MSI:X/MSA:X/S:X/AU:X/R:X/V:X/RE:X/U:X"
                    },
                    {
                        "key": "package_version",
                        "value": "2.8.4-r0"
                    },
                    {
                        "key": "package_name",
                        "value": "expat"
                    }
                ]
            },
            {
                "name": "CVE-2026-4775",
                "description": "A flaw was found in the libtiff library. A remote attacker could exploit a signed integer overflow vulnerability in the putcontig8bitYCbCr44tile function by providing a specially c
rafted TIFF file. This flaw can lead to an out-of-bounds heap write due to incorrect memory pointer calculations, potentially causing a denial of service (application crash) or arbitrary code execution.",
                "uri": "https://nvd.nist.gov/vuln/detail/CVE-2026-4775",
                "severity": "HIGH",
                "attributes": [
                    {
                        "key": "CVSS3_SCORE",
                        "value": "7.8"
                    },
                    {
                        "key": "CVSS3_VECTOR",
                        "value": "CVSS:3.1/AV:L/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:H"
                    },
                    {
                        "key": "package_version",
                        "value": "4.7.1-r0"
                    }
```




```
2a7ed9afc1c3: Layer already exists
379f51bf5b58: Layer already exists
74d97c428c51: Layer already exists
641b74319bab: Layer already exists
latest: digest: sha256:cc7f5196359e598814649deb5eea016f466edbe5dd1eae4dbb017770c5116dd7 size: 2196
+ docker logout
Removing login credentials for https://index.docker.io/v1/
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
SUCCESS: Image built, scanned, and pushed to Docker Hub.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```



```
+ set -eu
+ echo 'Scanning for HIGH and CRITICAL vulnerabilities...'
Scanning for HIGH and CRITICAL vulnerabilities...
+ /usr/bin/trivy --config '' --cache-dir /var/lib/jenkins/.cache/trivy image --severity HIGH,CRITICAL sunith18/jenkins-docker-lab:1
2026-10-14T16:02:36Z    INFO    [vulndb] Need to update DB
2026-10-14T16:02:36Z    INFO    [vulndb] Downloading vulnerability DB...
2026-10-14T16:02:36Z    INFO    [vulndb] Downloading artifact...        repo="mirror.gcr.io/aquasec/trivy-db:2"
24.86 MiB / 127.86 MiB [----------->_____________________________________________] 19.44% ? p/s ?46.48 MiB / 127.86 MiB [---------------------->_____________________________________] 36.35% ? p/s ?71.27 MiB / 127.86 MiB [--------------------------------->_____________________] 55.74% ? p/s ?98.73 MiB / 127.86 MiB [----------------------------------->__________] 77.21% 123.16 MiB p/s ETA 0s127.86 MiB / 127.86 MiB [----------------------------------------->] 100.00% 123.16 MiB p/s ETA 0s127.86 MiB / 127.86 MiB [----------------------------------------->] 100.00% 123.16 MiB p/s ETA 0s127.86 MiB / 127.86 MiB [----------------------------------------->] 100.00% 118.34 MiB p/s ETA 0s127.86 MiB / 127.86 MiB [----------------------------------------->] 100.00% 118.34 MiB p/s ETA 0s127.86 MiB / 127.86 MiB [----------------------------------------->] 100.00% 118.34 MiB p/s ETA 0s127.86 MiB / 127.86 MiB [----------------------------------------->] 100.00% 110.71 MiB p/s ETA 0s127.86 MiB / 127.86 MiB [----------------------------------------->] 100.00% 110.71 MiB p/s ETA 0s127.86 MiB / 127.86 MiB [----------------------------------------->] 100.00% 110.71 MiB p/s ETA 0s127.86 MiB / 127.86 MiB [----------------------------------------->] 100.00% 103.57 MiB p/s ETA 0s127.86 MiB / 127.86 MiB [----------------------------------------->] 100.00% 103.57 MiB p/s ETA 0s127.86 MiB / 127.86 MiB [----------------------------------------->] 100.00% 103.57 MiB p/s ETA 0s127.86 MiB / 127.86 MiB [----------------------------------------->] 100.00% 96.88 MiB p/s ETA 0s127.86 MiB / 127.86 MiB [----------------------------------------->] 100.00% 96.88 MiB p/s ETA 0s127.86 MiB / 127.86 MiB [----------------------------------------->] 100.00% 96.88 MiB p/s ETA 0s127.86 MiB / 127.86 MiB [----------------------------------------->] 100.00% 90.63 MiB p/s ETA 0s127.86 MiB / 127.86 MiB [----------------------------------------->] 100.00% 90.63 MiB p/s ETA 0s127.86 MiB / 127.86 MiB [----------------------------------------->] 100.00% 90.63 MiB p/s ETA 0s127.86 MiB / 127.86 MiB [----------------------------------------->] 100.00% 84.79 MiB p/s ETA 0s127.86 MiB / 127.86 MiB [----------------------------------------->] 100.00% 29.12 MiB p/s 4.6s2026-10-14T16:02:41Z        INFO    [vulndb] Artifact successfully downloaded       repo="mirror.gcr.io/aquasec/trivy-db:2"
2026-10-14T16:02:41Z    INFO    [vuln] Vulnerability scanning is enabled
2026-10-14T16:02:41Z    INFO    [secret] Secret scanning is enabled
2026-10-14T16:02:41Z    INFO    [secret] If your scanning is slow, please try '--scanners vuln' to disable secret scanning
2026-10-14T16:02:41Z    INFO    [secret] Please see https://trivy.dev/docs/v0.76/guide/scanner/secret#recommendation for faster secret detection
2026-10-14T16:02:51Z    INFO    Detected OS     family="alpine" version="3.24.2"
2026-10-14T16:02:51Z    INFO    [alpine] Detecting vulnerabilities...   os_version="3.24" repository="3.24" pkg_num=71
2026-10-14T16:02:51Z    INFO    Number of language-specific files       num=0
2026-10-14T16:02:51Z    WARN    Using severities from other vendors for some vulnerabilities. Read https://trivy.dev/docs/v0.76/guide/scanner/vulnerability#severity-selection for details.
```



```
sunith18/jenkins-docker-lab:1 (alpine 3.24.2)
==============================================
Total: 3 (HIGH: 3, CRITICAL: 0)

┌─────────┬────────────────┬──────────┬────────┬───────────────────┬───────────────┬──────────────────────────────────────────────────────────┐
│ Library │ Vulnerability  │ Severity │ Status │ Installed Version │ Fixed Version │                          Title                           │
├─────────┼────────────────┼──────────┼────────┼───────────────────┼───────────────┼──────────────────────────────────────────────────────────┤
│ libexpat │ CVE-2026-93990 │ HIGH     │ fixed  │ 2.8.4-r0          │ 2.8.5-r0      │ expat: Expat: XML Injection via Malformed UTF-16 Input   │
│         │                │          │        │                   │               │ https://avd.aquasec.com/nvd/cve-2026-93990               │
├─────────┼────────────────┼──────────┼────────┼───────────────────┼───────────────┼──────────────────────────────────────────────────────────┤
│ pcre2   │ CVE-2026-103111 │          │        │ 10.48-r0          │ 10.49-r0      │ pcre2: pcre2: Out-of-bounds write via crafted regular    │
│         │                │          │        │                   │               │ expression                                               │
│         │                │          │        │                   │               │ https://avd.aquasec.com/nvd/cve-2026-103111              │
├─────────┼────────────────┼──────────┼────────┼───────────────────┼───────────────┼──────────────────────────────────────────────────────────┤
│ tiff    │ CVE-2026-4775  │          │        │ 4.7.1-r0          │ 4.7.2-r0      │ libtiff: libtiff: Arbitrary code execution or denial of  │
│         │                │          │        │                   │               │ service via signed integer...                            │
│         │                │          │        │                   │               │ https://avd.aquasec.com/nvd/cve-2026-4775                │
└─────────┴────────────────┴──────────┴────────┴───────────────────┴───────────────┴──────────────────────────────────────────────────────────┘
+ echo 'Blocking the push only if CRITICAL vulnerabilities exist...'
Blocking the push only if CRITICAL vulnerabilities exist...
+ /usr/bin/trivy --config '' --cache-dir /var/lib/jenkins/.cache/trivy image --severity CRITICAL --exit-code 1 sunith18/jenkins-docker-lab:1
2026-10-14T16:02:51Z    INFO    [vuln] Vulnerability scanning is enabled
2026-10-14T16:02:51Z    INFO    [secret] Secret scanning is enabled
2026-10-14T16:02:51Z    INFO    [secret] If your scanning is slow, please try '--scanners vuln' to disable secret scanning
2026-10-14T16:02:51Z    INFO    [secret] Please see https://trivy.dev/docs/v0.76/guide/scanner/secret#recommendation for faster secret detection
2026-10-14T16:02:51Z    INFO    Detected OS     family="alpine" version="3.24.2"
2026-10-14T16:02:51Z    INFO    [alpine] Detecting vulnerabilities...   os_version="3.24" repository="3.24" pkg_num=71
2026-10-14T16:02:51Z    INFO    Number of language-specific files       num=0
2026-10-14T16:02:51Z    WARN    Using severities from other vendors for some vulnerabilities. Read https://trivy.dev/docs/v0.76/guide/scanner/vulnerability#severity-selection for details.

Report Summary

┌──────────────────────────────────────┬──────┬─────────────────┬─────────┐
│                Target                │ Type │ Vulnerabilities │ Secrets │
├──────────────────────────────────────┼──────┼─────────────────┼─────────┤
│ sunith18/jenkins-docker-lab:1 (alpine 3.24.2) │ alpine │        0        │    -    │
└──────────────────────────────────────┴──────┴─────────────────┴─────────┘
```

8. Conclusion

This lab demonstrates a basic CI/CD workflow for container images using Jenkins, Docker, Trivy, and Docker Hub. It establishes an automated path from image creation through vulnerability scanning to registry publishing. Further improvements can include stricter vulnerability thresholds, base-image updates, image signing, and isolated Jenkins agents.